

EREDITARIETÀ: PRIMA PARTE

L'ereditarietà è un costrutto sintattico, a livello di linguaggio, che consente ad una classe di derivare un'altra classe.

La classe derivata (ovvero la classe che specializza la classe più generica) eredita le variabili e i metodi della classe base, detta anche **classe madre**. La classe figlia, oltre ad ereditare ciò che proviene dalla classe base, aggiunge i suoi propri metodi e le sue proprie variabili.

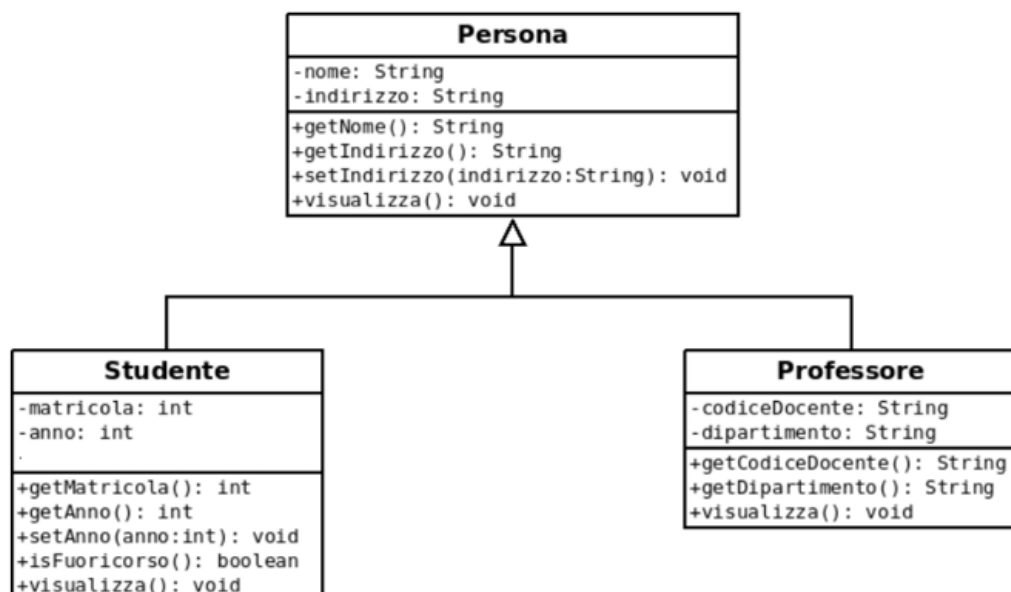
L'elemento chiave nell'introduzione di questo concetto è il **riuso del codice**.

Esempio: si ha una classe Madre che rappresenta una famiglia di concetti, i quali vengono concretizzati dalle classi che la specializzano. Le classi derivate, specializzando il concetto iniziale, non ripetono il codice del 'genitore', ma lo ereditano. Di conseguenza, esiste una porzione di codice condivisa da tutte le classi figlie, che quindi basta modificare una sola volta.

L'ereditarietà consente, quando si programma, di fare riferimento al mondo reale. Questo perché l'ereditarietà riflette nelle relazioni gerarchiche che si incontrano nella realtà.

L'ereditarietà garantisce, inoltre, l'**estensibilità del codice**: è possibile estendere il comportamento di una classe esistente senza modificare il codice originale, aggiungendo nuove funzionalità tramite le sottoclassi.

L'ereditarietà facilita la manutenzione del codice: miglioramenti e modifiche possono essere apportati in una classe base, e queste modifiche saranno automaticamente propagate a tutte le sottoclassi che ereditano da essa.



Sintassi linguaggio Java per implementare l'ereditarietà:

```
public class Persona{
    private String nome;
    private String indirizzo;
    //Costruttori e metodi
    ...
    ...
}
public class Studente extends Persona{
    ...
    ...
}
Public class Professore extends Persona{
    ...
    ...
}
```

Significato di **extends**: la classe figlia (o le classi figlie) eredita attributi e metodi. La classe Studente e la classe Professore sono un'estensione della classe Persona: oltre agli attributi ai metodi ereditati da Persona, possono avere altri attributi e altri metodi che le qualificano e le specializzano.

POLIMORFISMO NELLA PROGRAMMAZIONE OOP

Il termine polimorfismo deriva dal greco antico. È composto dalla parola *poly*, che significa molti e *morphe*, che significa forma.

In generale polimorfismo indica la capacità di assumere più forme. Nella programmazione orientata agli oggetti è strettamente legato all'ereditarietà e alla ridefinizione dei metodi.

Esistono due principali tipi di polimorfismo: **polimorfismo di sovraccarico** (overloading) e **polimorfismo di inclusione** (overriding).

OVERLOADING

L'overloading è un tipo di polimorfismo che consente a più metodi di una stessa classe di avere lo stesso nome, ma con un numero o un tipo diverso di parametri.

Il compilatore, comparando il numero e il tipo dei parametri, riesce a individuare qual è il metodo corretto da invocare.

Un esempio di metodo che utilizza l'overloading è ***println***. Questo metodo consente di stampare sullo schermo i diversi parametri che vengono passati, tra cui stringhe, numeri interi e reali.

Esempio:

```
System.out.println("Stringa di testo");
System.out.println(23);
System.out.println(5.896);
```

L'overloading fornisce un modo per far assumere allo stesso metodo dei comportamenti diversi: a seconda dei parametri che vengono passati, viene eseguito un comportamento diverso.

L'overloading è usato per offrire più di un costruttore ad una classe.

Esempio:

```
public class Numero{
    private int num;
    public Numero() //costruttore non parametrizzato
    {
        num = 0;
    }
}
```

```

    }
    public Numero(int num) //costruttore parametrizzato
    {
        this.num = num;
    }
}

```

La scelta di quale dei due metodi richiamare viene effettuata dal compilatore in base alla corrispondenza dei parametri.

```
Numero n1 = new Numero();
```

```
Numero n2 = new Numero(100);
```

OVERRIDING

L'overriding è un tipo di polimorfismo che consente ad una sottoclasse (classe derivata) di fornire una specifica implementazione di un metodo già definito nella classe madre (SUPERCLASSE).

Esempio:

```

class Pagamento {
    private double importo;
    public Pagamento(double importo) {
        this.importo = importo;
    }
    public double getImporto() {
        return importo;
    }
    public void effettuaPagamento() {
        System.out.println("Pagamento di " + importo + " €
effettuato");
    }
}

```

```
class PagamentoCartaCredito extends Pagamento {
    private String numeroCarta;

    public PagamentoCartaCredito(double importo, String
numeroCarta) {
        super(importo);
        this.numeroCarta = numeroCarta;
    }

    public String getNumeroCarta() {
        return numeroCarta;
    }

    public void effettuaPagamento() {
        System.out.println("Pagamento di " + getImporto() + " €
con carta di credito (" + numeroCarta + ") effettuato");
    }
}
```

```
class PagamentoPayPal extends Pagamento {
    private String email;

    public PagamentoPayPal(double importo, String email) {
        super(importo);
        this.email = email;
    }

    public String getEmail() {
        return email;
    }

    public void effettuaPagamento() {
        System.out.println("Pagamento di " + getImporto() + " €
con PayPal (" + email + ") effettuato");
    }
}
```

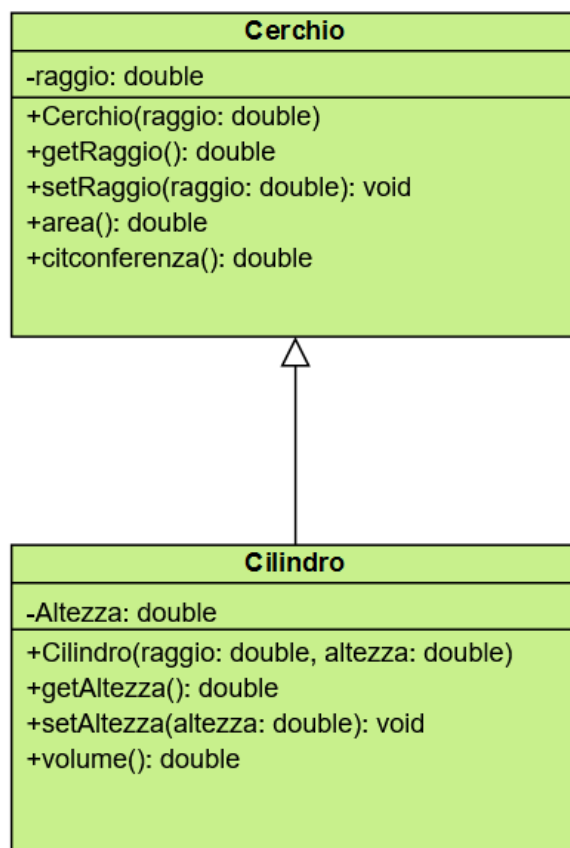
```
public class Main {
    public static void main(String[] args) {
        Pagamento mioPagamentoCC = new
PagamentoCartaCredito(100.0, "1234-5678-9876-5432");

        Pagamento mioPagamentoPP = new PagamentoPayPal(200.0,
"nominativo@example.com");

        mioPagamentoCC.effettuaPagamento();
        mioPagamentoPP.effettuaPagamento();
    }
}
```

Nota bene: i metodi setter non sono stati implementati.

Ereditarietà: calcolo del volume del cilindro.



```
public class Cerchio {
    private double raggio;

    public Cerchio(double raggio)
    {
        setRaggio(raggio);
    }
    public double getRaggio()
    {
        return raggio;
    }
    public void setRaggio(double raggio)
    {
        if(raggio>0)
        {
            this.raggio=raggio;
        }
        else
        {
            throw new IllegalArgumentException("Il valore del raggio deve essere maggiore di 0.");
        }
    }
    public double area()
    {
        return (raggio*raggio*Math.PI);
    }
    public double circonferenza()
    {
        return (2*raggio*Math.PI);
    }
}
```

```
public class Cilindro extends Cerchio {
    private double altezza;

    public Cilindro(double raggio, double altezza)
    {
        super(raggio);
        setAltezza(altezza);
    }
    public double getAltezza()
    {
        return altezza;
    }
    public void setAltezza(double altezza)
    {
        if(altezza>0)
        {
            this.altezza=altezza;
        }
        else
        {
            throw new IllegalArgumentException("Il valore dell'altezza deve essere maggiore di 0.");
        }
    }
    public double volume()
    {
        double vol=area()*altezza;
        return vol;
    }
}
```



```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double raggio;
        double altezza;
        boolean successo=false;
        do {
            try {

                System.out.print("Inserisci il valore del raggio (cm): ");
                raggio = Double.parseDouble(scanner.nextLine());
                System.out.print("Inserisci il valore dell'altezza (cm): ");
                altezza = Double.parseDouble(scanner.nextLine());

                // Creazione del cilindro con valori validi
                Cilindro cil = new Cilindro(raggio, altezza);
                System.out.println("Cilindro creato con raggio " + cil.getRaggio() + " e altezza " + cil.getAltezza());
                System.out.println("Area di base: " + cil.area() + " cm^2");
                System.out.println("Volume: " + cil.volume() + " cm^3");
                successo=true;
            }
            catch (NumberFormatException e) {
                System.out.println("Errore nell'inserimento: inserisci un numero valido.");
            }
            catch (IllegalArgumentException e) {
                System.out.println("Errore: " + e.getMessage());
            }
        }
        while(!successo);
        scanner.close();
    }
}

```